MAKERERE UNIVERSITY

COLLEGE OF COMPUTING AND INFORMATION SCIENCES

COURSE NAME: SYSTEM INTEGRATION & DEPLOYMENT

COURSE CODE: IST 3206

GROUP FIVE (5)		
NAME	**REG. NO**	**STUDENTS NO.**
OKWIR MOSES	23/U/16751/PS	2300716751
KAU VINCENT JAMES	23/U/0547	2300700547
NKULIKIYUMUKIZA JEAN D'AMOUR	23/X/25120/PS	2300725120

System Design Document (SDD) For the Timetable Calendar Synchronization System

Table of Contents

1. Introduction 1

1.1 Purpose of the Document 1

1.2 System Overview 1

1.3 Technology Stack 1

2. System Architecture 2

2.1 Architectural Design 2

3. Detailed System Flow 4

3.1 Subscription Phase 4

3.2 Synchronization Phase 5

4. Database Design 7

4.1 Entity Identification and Attributes 7

4.2 Relationships and Cardinality 8

4.3 Data Integrity & Normalization 8

5. System Modeling (UML) 9

5.1 Class Diagram 9

5.2 Sequence Diagram 12

5.3 Use Case Diagram 14

6. Security Design 15

7. Performance and Scalability 16

List of Figures

Figure 1 - High-level Architecture Design 4
Figure 2 - System Flowchart 6
Figure 3 - Entity-Relationship Diagram 9
Figure 4 - Class Diagram 12
Figure 5 - Sequence Diagram 14
Figure 6 - Use Case Diagram 15

1. Introduction

1.1 Purpose of the Document

This Software Design Document presents the detailed design of the Timetable Calendar Synchronization System. It serves as a bridge between the system requirements and the actual implementation by describing how the system will be structured, how its components will interact, and how data will flow throughout the system.

The document is intended for software developers, system designers, and technical stakeholders who will be involved in the implementation and maintenance of the system. It ensures that all parties have a shared understanding of how the system will be built.

1.2 System Overview

The system is designed to integrate the Makerere University Timetabling Management System with widely used calendar platforms such as Google Calendar and Microsoft Outlook. Its primary function is to automatically convert timetable data into calendar events and synchronize them with student calendars using the iCalendar (ICS) standard.

Instead of requiring students to manually check their timetables, the system ensures that all schedule information is readily available in their personal calendars and updated automatically whenever changes occur.

1.3 Technology Stack

The system will be implemented using the PERN stack, which consists of PostgreSQL, Express.js, React.js, and Node.js. This stack is selected because it provides a full JavaScript-based development environment, enabling consistency across both frontend and backend development.

The frontend will be developed using React.js, providing a responsive and interactive user interface. This is particularly important for implementing the “Add to Calendar” feature in a simple and user-friendly way.

The backend will be implemented using Node.js with Express.js. Express will handle API routing, request processing, and communication between the frontend and the database. It will also manage the logic for generating and updating calendar feeds.

The database will be PostgreSQL, which will store structured data such as student records, timetable information, and generated tokens. PostgreSQL is chosen for its reliability, strong relational capabilities, and support for complex queries.

The system will expose RESTful APIs for communication between components and will generate calendar data using the iCalendar (ICS) format, ensuring compatibility with external calendar applications such as Google Calendar and Microsoft Outlook.

2. System Architecture

2.1 Architectural Design

The Timetable Calendar Synchronization System is built using an Integrated Middleware Architecture (also known as a Plug-in or Service-Oriented model). In this design, the system acts as a specialized functional extension that "plugs into" the existing Makerere Timetabling Management System to provide automated synchronization without replacing the core university infrastructure.

Detailed System Flow

The following steps describe how data and requests move through the layers of the architecture:

Step 1: The User Trigger (Integration Point)

The process begins at the Presentation Layer, which is hosted directly within the Makerere Timetabling Management System UI. A student clicks the "Add to Calendar" button, which sends a secure HTTPS request containing the student's identity to your system's API Gateway.

Step 2: Tokenization and Secure Storage

The API Gateway (Express.js) receives the request and communicates with the internal PostgreSQL Database. The system generates a unique, long-lived secure token for the student and stores it in the database. This token allows the student to access their calendar feed privately without needing to log in repeatedly.

Step 3: Identity Mapping and Orchestration

When a calendar application (like Google Calendar) requests an update, the Calendar Sync Service acts as the orchestrator, queries the PostgreSQL DB to perform a Token-to-Identity

Mapping. This step identifies which specific student registration number corresponds to the token provided by the calendar app.

Step 4: Real-Time Data Retrieval

The Calendar Sync Service reaches out to the external Makerere Timetabling Management System. It retrieves the "Source of Truth" data, the most current lecture times, course names, lecturer, course code and lecture rooms directly from the university's database.

Step 5: Data Transformation

The raw timetable data is passed internally from the Sync Service to the ICS Feed Generator, which transforms it into the iCalendar (ICS) format (RFC 5545), ensuring compatibility with all major calendar platforms.

Step 6: Delivery and Subscription

The ICS Feed Generator publishes the formatted calendar data as a live URL feed. To external calendars, such as Google Calendar and Microsoft Outlook, to subscribe and automatically download updates whenever changes are detected in the university schedule.

Component Summary Table

Component	**Layer**	**Primary Role**
Makerere UI	Presentation	Hosts the "Add to Calendar" trigger for students.
API Gateway	Application	Manages entry security and Token Generation.
Sync Service	Application	Orchestrates Identity Mapping and fetches external data.
ICS Generator	Application	Performs data transformation into iCalendar format.
PostgreSQL DB	Data (Internal)	Stores secure tokens and student mapping metadata.

Diagram

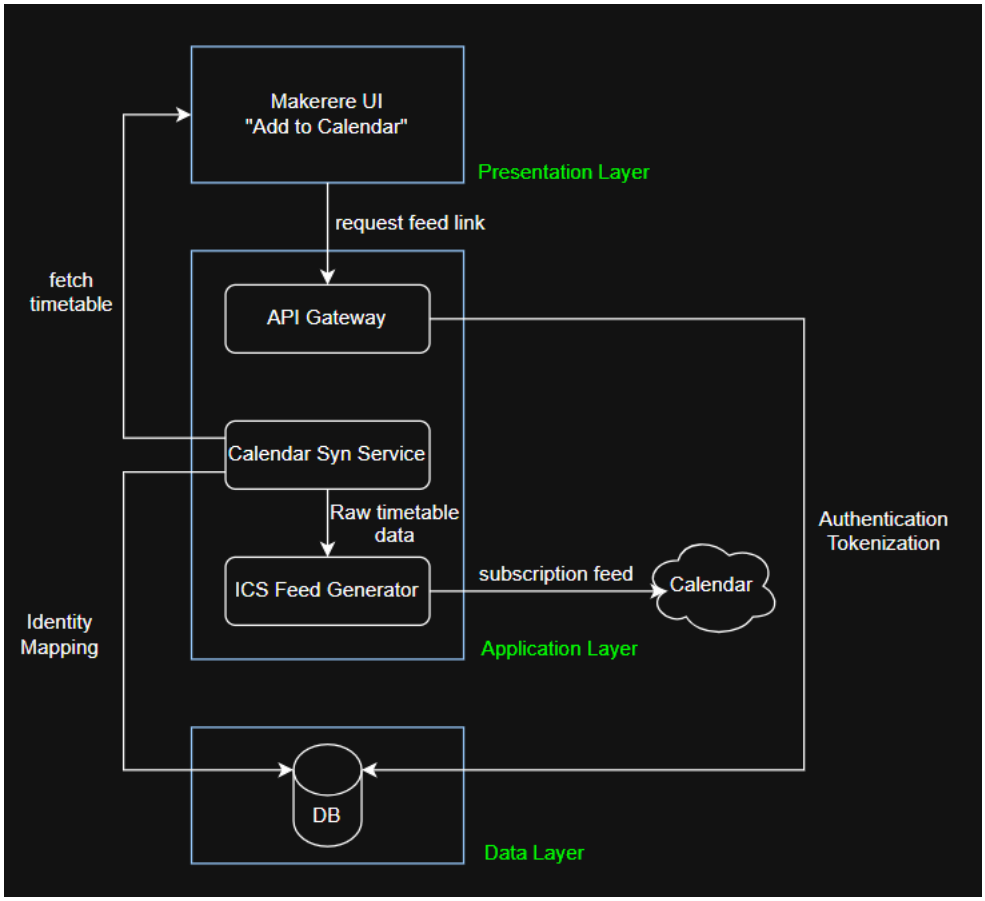

Figure 1 - High-level Architecture Design

3. Detailed System Flow

The Timetable Calendar Synchronization System operates through a two-phase process: the Subscription Phase, where a student registers for a feed, and the Synchronization Phase, where the calendar application automatically fetches updates. The following flowchart and detailed step descriptions outline this logic.

3.1 Subscription Phase

This phase occurs when a student interacts with the university portal to initiate the link between their timetable and their personal device.

Step 1 - Start & Access: The process begins when a student accesses the Makerere Timetabling Management System. The student navigates to their specific course schedule view.

Step 2 - Integration Trigger: Within the timetable interface, the student clicks the "Add to Calendar" button. This action triggers a request from the Makerere User Interface (UI) to the Synchronization System's API Gateway.

Step 3 - Identity Transmission: The Makerere UI sends the student's unique identification (Registration Number) and a session authorization token to the API Gateway via a secure HTTPS POST request.

Step 4 - Tokenization Logic (Decision): The API Gateway queries the PostgreSQL Database to check if a secure calendar token already exists for this student.

- **If No:** The system generates a new, cryptographically secure 64-character token and stores it in the database alongside the student's registration number.
- **If Yes:** The system retrieves the existing token to ensure the student maintains a consistent feed URL across devices.

Step 5 - URL Delivery: The system constructs a unique subscription URL (for example, https://sync.mak.ac.ug/feed/23.U.11.PS) and returns it to the Makerere UI. The student is presented with a "Copy Link" option or a "Subscribe" button.

3.2 Synchronization Phase

Once the student has added the URL to their calendar app (Google Calendar or Microsoft Outlook), the following steps occur automatically in the background without further user intervention.

Step 6 - Feed Request: At periodic intervals (determined by the calendar provider), Google or Outlook sends an HTTPS GET request to the unique URL provided in Step 5.

Step 7 - Token Validation: The Sync Service receives the request and extracts the token. It validates this token against the PostgreSQL Database to identify the student and ensure the request is authorized.

Step 8 - Real-Time Data Fetching: After identifying the student, the Sync Service initiates a secure connection to the Makerere Timetabling Management System. It fetches the latest "Source of Truth" schedule data, including Course Code and Title, Lecture Start and End Times, Assigned Lecturer, and Lecture Room.

Step 9 - iCalendar Transformation: The raw data is passed to the ICS Feed Generator. This component maps the university's data fields into the RFC 5545 (iCalendar) format.

Step 10: Response & End: The system streams the generated .ics file back to the calendar application. The student's personal device processes this file, and the updated lecture schedule becomes visible on their mobile or desktop calendar.

Diagram

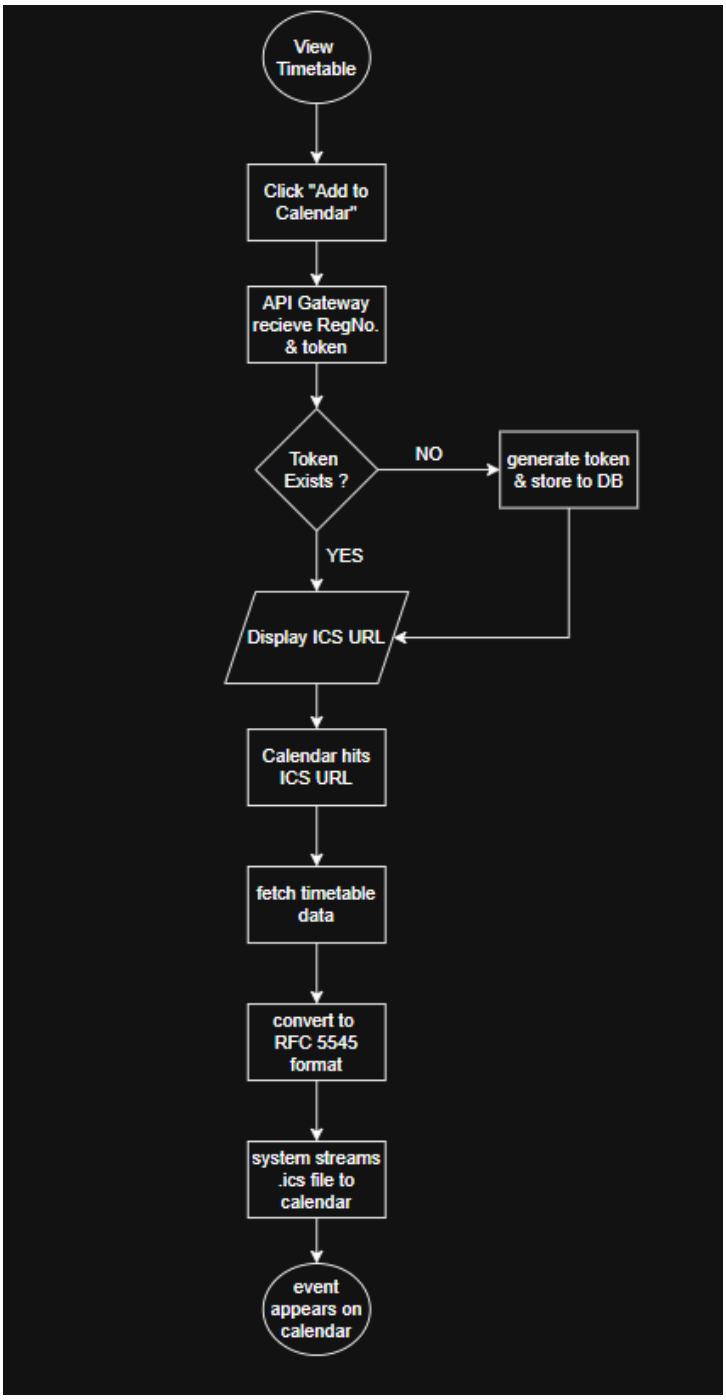

Figure 2 - System Flowchart

4. Database Design

For an Integrated Middleware Architecture, the database remains lightweight. Since the actual timetable data is fetched in real-time from the Makerere system, our PostgreSQL database serves as the "Security and Identity Mapping" layer.

4.1 Entity Identification and Attributes

1. Entity: Students

Represents the registered users of the system. It acts as the primary anchor for all synchronization data.

Attributes:

- student_id (PK): A unique internal identifier (UUID or Serial) for each student.
- reg_number: The official university registration number, used as a unique string to query the external Makerere Timetabling System.
- name: The full name of the student for personalization and verification.
- email: The student's official university email address for account-related notifications.

2. Entity: Tokens

Manages the secure, unique feed URLs generated for the students. This entity ensures that timetable access is kept private.

Attributes:

- token_id (PK): A unique identifier for the specific feed link.
- student_id (FK): Links the token to a specific record in the Students table.
- secure_hash: A cryptographically secure, unique 64-character string that forms the "secret" part of the ICS URL.
- created_at: A timestamp recording when the subscription link was generated.
- last_accessed: A timestamp updated every time an external calendar application (Google/Outlook) polls the URL.

3. Entity: SyncLogs

A diagnostic entity used for auditing and troubleshooting the synchronization process.

Attributes:

- log_id (PK): A unique identifier for each log entry.
- token_id (FK): Links the log entry to the specific Token used during the sync attempt.
- status: An enumeration ('SUCCESS' or 'FAILED') indicating the result of the data fetch and transformation.
- timestamp: The exact date and time the synchronization was attempted.
- error_details: A text field capturing specific error messages (for example, "Makerere DB Timeout" or "Empty Schedule").

4.2 Relationships and Cardinality

1. Students to Tokens (1:N)

Relationship: Has

Cardinality: One-to-Many (1:N). One student may generate multiple tokens (for example, if they wish to refresh their security or use different links for different devices), but each token belongs to exactly one student.

Constraint: Deleting a student record will trigger a cascade delete of all associated tokens.

2. Tokens to SyncLogs (1:N)

Relationship: Generates

Cardinality: One-to-Many (1:N). A single unique token will generate many log entries over its lifetime, as external calendar applications typically poll the URL every 1–2 hours. Every log entry, however, is tied to the specific token used to initiate that request.

Benefit: This relationship allows for granular debugging, enabling administrators to identify if a failure is specific to a particular device or subscription link rather than the student's account as a whole.

4.3 Data Integrity & Normalization

The database is designed to **Third Normal Form (3NF)** to eliminate data redundancy. By linking SyncLogs to Tokens rather than directly to Students, the system maintains a clean path

of dependency: a log exists because a token was used, and a token exists because a student requested it. This ensures that the system remains scalable and that the logic for security (Tokens) is logically separated from the logic for auditing (Logs).

Diagram

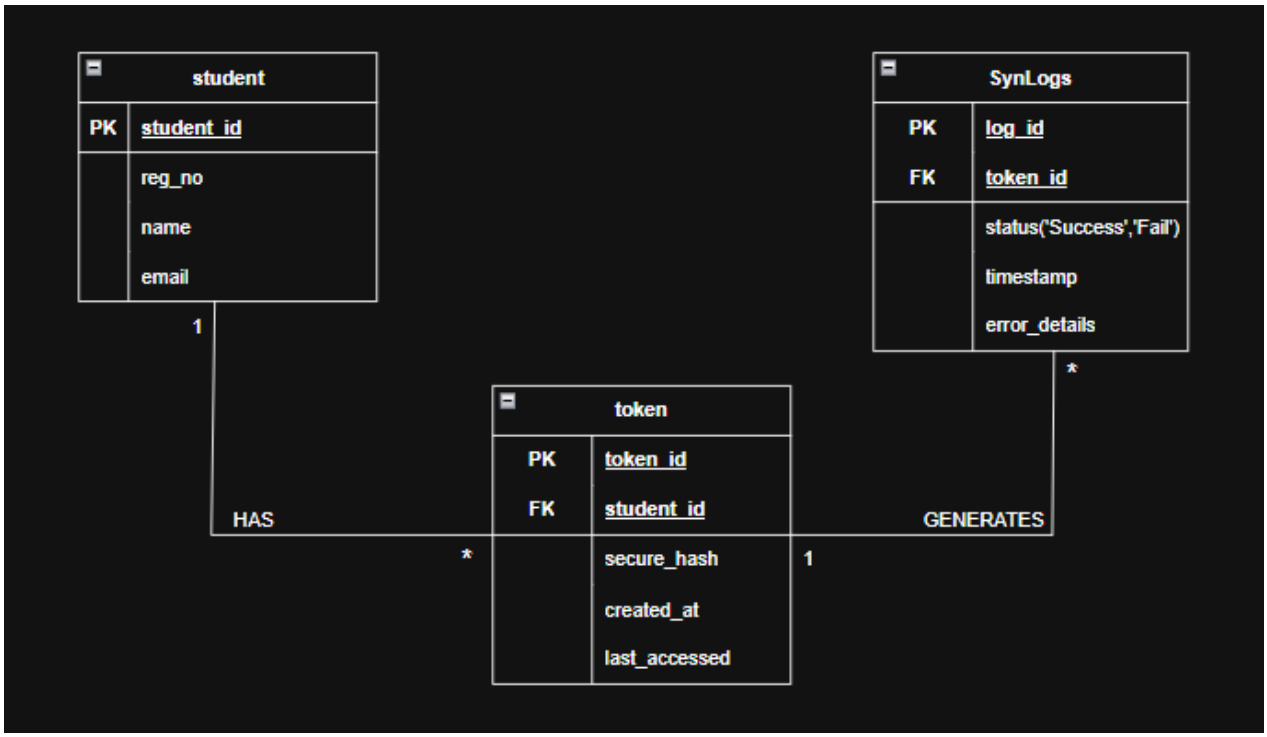

Figure 3 - Entity-Relationship Diagram

5. System Modeling (UML)

5.1 Class Diagram

The Class Diagram defines the object-oriented structure of the middleware, illustrating how the PERN stack components interact to handle student data, security, and external synchronization.

5.1.1 Entity Class Definitions (Model Layer)

These classes represent the data structure of the system and mirror the PostgreSQL database schema.

1. Class: Student

Attributes:

- student_id (int): Primary internal identifier.

- reg_number (string): The university-issued registration number.
- name (string): Full name of the student.
- email (string): Student's university contact email.

Methods: + getProfile()

2. Class: Token

Attributes:

- token_id (int): Primary key for the security link.
- student_id (int): Foreign key connecting to the Student.
- secure_hash (string): The unique 64-character token used in the URL.
- created_at (timestamp): Date of generation.
- last_accessed (timestamp): Date of the last calendar sync request.

Methods: + isValid(): boolean

3. Class: SyncLog

Attributes:

- log_id (int): Primary key for the audit entry.
- token_id (int): Foreign key connecting to the specific Token used.
- status (string): Result of the sync (e.g., "SUCCESS", "FAILED").
- timestamp (timestamp): Exact time of the synchronization attempt.
- error_details (string): Capture of system errors if the sync fails.

Methods: + createEntry()

5.1.2 Logic & Boundary Class Definitions

4. Class: CalendarController (Boundary)

This manages HTTP traffic and API endpoints.

Methods:

- + handleSubscription(): Processes the initial "Add to Calendar" request.
- + serveFeed(secure_hash): Handles the recurring .ics file requests from external apps.

5. Class: SyncService (Service)

This is the primary orchestrator of data movement and identity mapping.

Methods:

- + fetchRemoteTimetable(reg_number): Queries the Makerere DB.
- + mapIdentity(secure_hash): Resolves which student owns the provided token.

6. Class: ICalGenerator (Utility)

This standardizes the outgoing data format.

Methods:

- + convertToICS(data): string: Transforms raw rows into RFC 5545 format.

5.1.3 Relationship & Connection Logic

The diagram uses specific UML notation to distinguish between structural ownership and functional usage.

1. Structural Associations (Solid Lines)

Student to Token: This is connected with a solid line having the cardinality of 1 to 0..* (One student can have zero or many tokens). This is a permanent relationship where the student "owns" the security tokens.

Token to SyncLog: Connected with a solid line and having the cardinality of 1 to * (One token generates many log entries over time). This allows for granular debugging of specific subscription links.

2. Functional Dependencies (Dashed Arrows)

In this system, we use dashed arrows to represent "Usage" or "Dependency." This indicates that one class requires the functionality of another temporarily to complete a task, but does not "own" it.

CalendarController to SyncService: Connection is established with a dashed arrow and labelled “calls”. The Controller receives the web request and triggers the Service to perform the sync. It is a "request-response" dependency.

SyncService to ICalGenerator: A dashed arrow is used to establish a connection with the label “formats with”. The SyncService retrieves raw data but relies on the Generator class to transform that data into a valid .ics file.

SyncService to SyncLog: Connection is mapped using a dashed arrow and labelled “creates”. After every synchronization attempt, the SyncService creates a new Log entry object to record the status. The service "writes" the log, but it is not a permanent part of the log's data structure.

Diagram

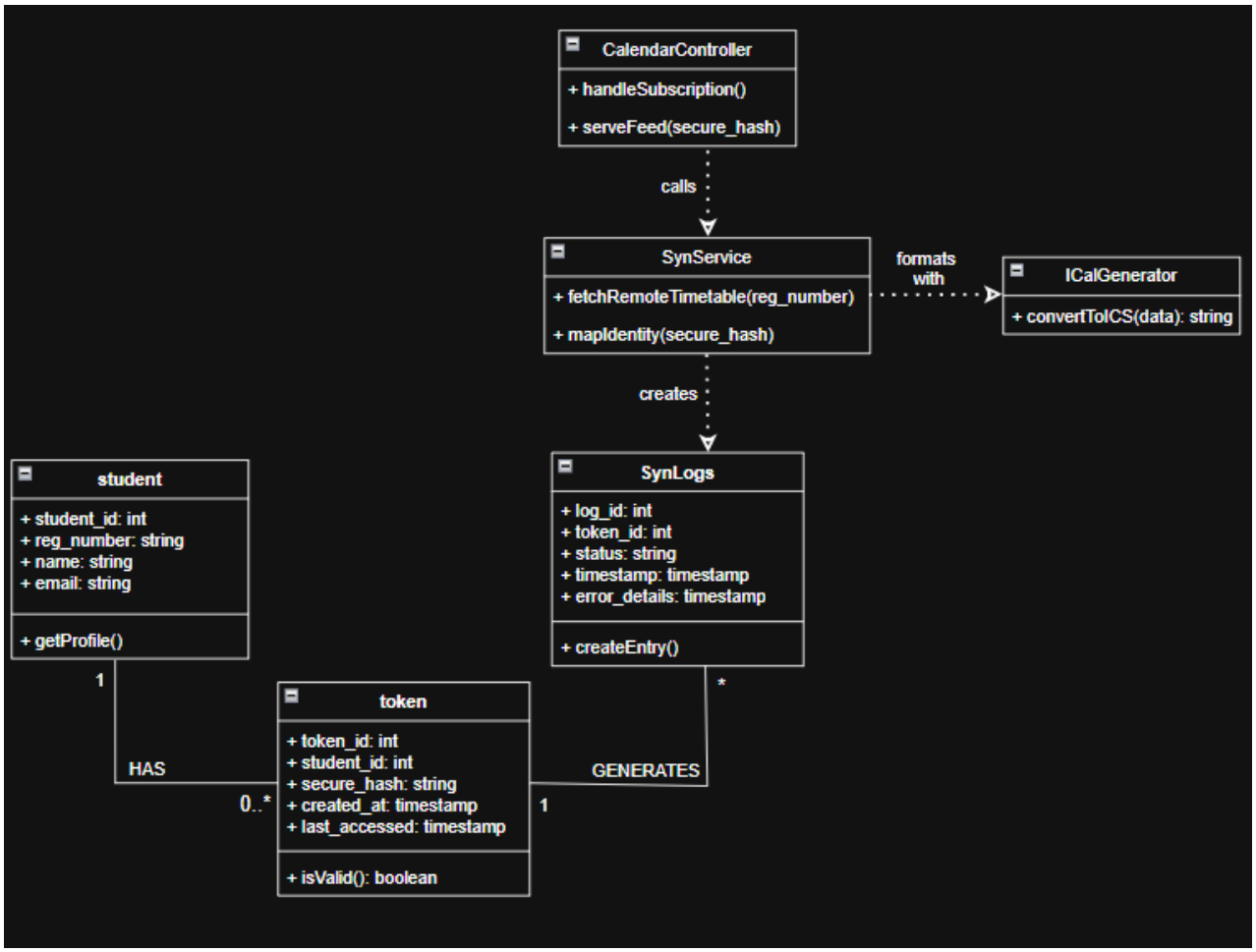

Figure 4 - Class Diagram

5.2 Sequence Diagram

The Sequence Diagram illustrates the time-ordered interaction between the middleware components and external entities during a background synchronization event. This process is

entirely automated and occurs whenever an external calendar provider (Google Calendar) polls the system's unique URL.

5.2.1 Message Flow Breakdown

Request Initiation: The process begins with an external Calendar App sending an HTTPS GET request containing the student's unique secure_hash.

Security & Mapping: The CalendarController delegates the request to the SyncService, which queries the internal PostgreSQL DB. This step is critical as it maps the anonymized hash back to the student's official reg_number without exposing the identity in the URL.

External Integration: Once the identity is resolved, the system performs a real-time fetch from the Makerere Timetabling DB. This ensures that any last-minute venue changes or cancelled lectures are reflected immediately in the student's calendar.

Data Serialization: The raw data is passed to the ICalGenerator, which serializes the academic information into the RFC 5545 standard.

Audit Logging: Simultaneously, the SyncService instantiates a SyncLog entry. This ensures that even if the delivery is successful, the system maintains a record of the transaction for performance monitoring.

Response Delivery: Finally, the generated iCalendar file is streamed back to the requesting application, completing the sync cycle.

Diagram

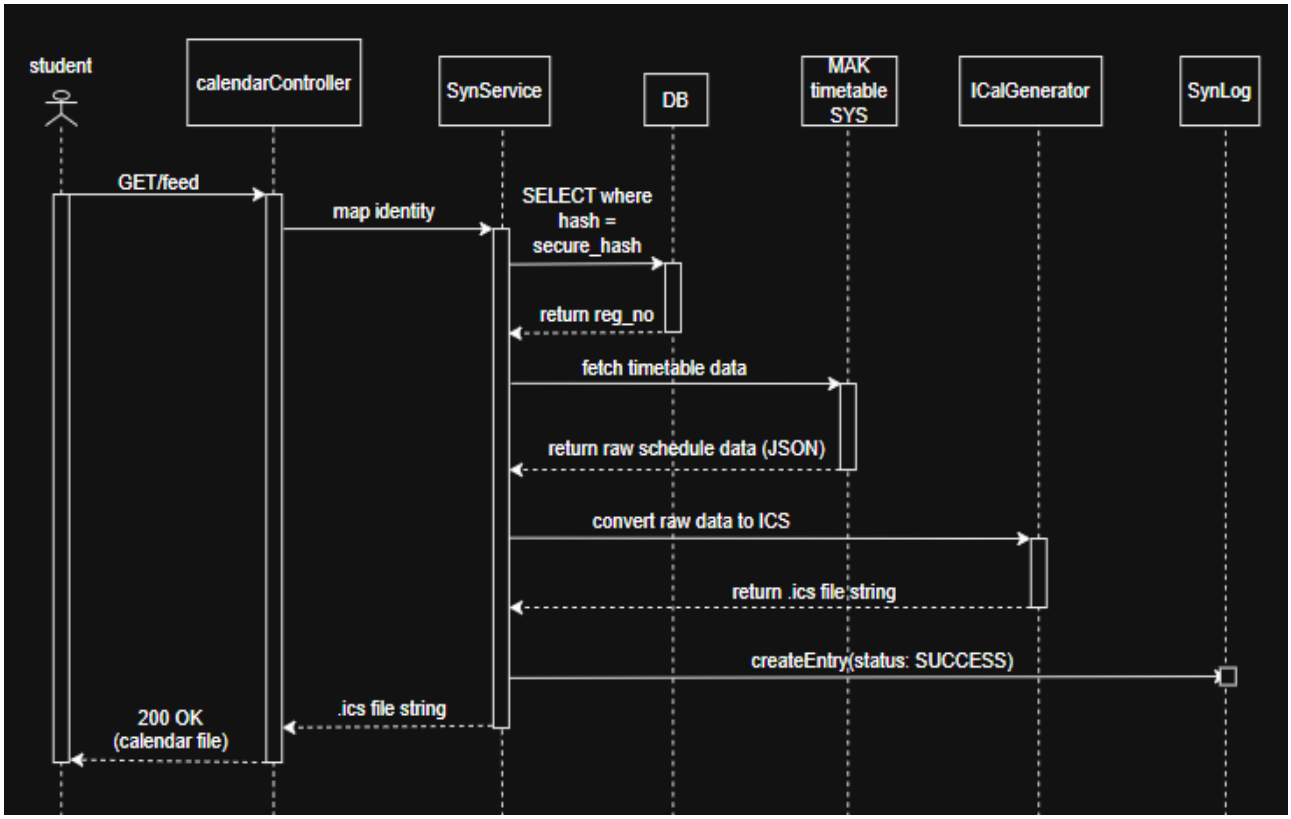

Figure 5 - Sequence Diagram

5.3 Use Case Diagram

The Use Case Diagram defines the functional scope of the system and the interactions between human actors and external technical entities. It highlights the boundary between the middleware and the existing university infrastructure.

5.3.1 Primary Actors and Use Cases

Student: The primary beneficiary of the system. The student interacts with the system to initiate the synchronization process by generating a unique, secure URL through the Makerere User Interface.

External Calendar App (Google/Outlook): Acts as a secondary actor that performs recurring requests. It is responsible for consuming the generated ICS feed to keep the student's personal device updated.

Makerere Timetabling System: An external system actor that serves as the "Source of Truth." The middleware depends on this system to provide real-time schedule updates.

5.3.2 Functional Relationships

The diagram identifies a critical <<include>> relationship between the "Download ICS Feed" and "Fetch Timetable Data" use cases. This ensures that every time a calendar application requests a sync, the middleware performs a real-time retrieval from the university database to ensure data accuracy. The use of a clear system boundary illustrates that while the system is "integrated" into Makerere, it operates as a distinct functional entity managing its own security tokens and transformation logic.

Diagram

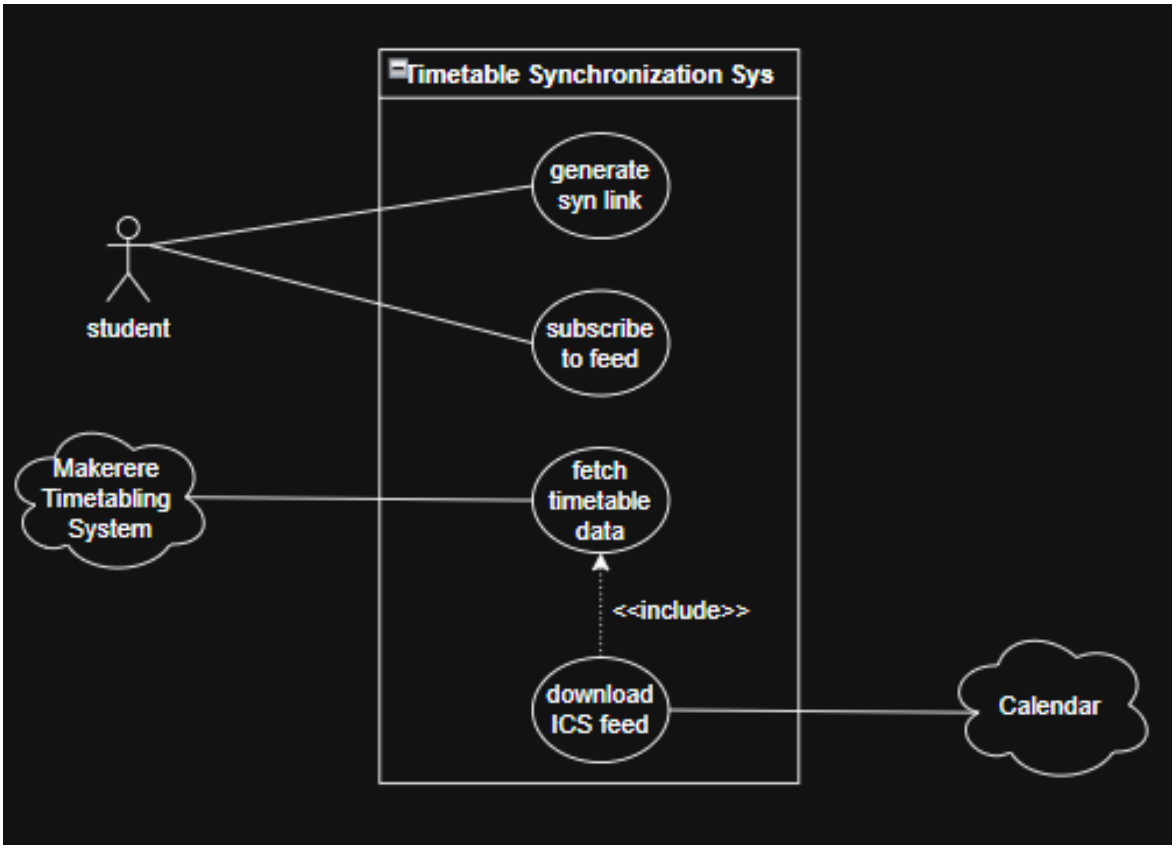

Figure 6 - Use Case Diagram

6. Security Design

Security is a critical aspect of the system, particularly because it involves personal academic schedules.

Each calendar feed is protected using a unique token that is difficult to guess. This ensures that only authorized users can access the feed.

Additionally, all communication between the system and external services is encrypted using HTTPS. This prevents unauthorized interception of data.

7. Performance and Scalability

The system is designed to handle a large number of users efficiently. Techniques such as caching will be used to reduce the need for repeated data processing.

The use of a scalable backend framework ensures that the system can grow as the number of users increases without significant performance degradation.